Flow Matching & GRPO

E-568 Theory and Methods of Reinforcement Learning taught at EPFL by Prof. Volkan Cevher

PREPARED BY

Bohdan Naida

Vu Nguyen

OUTLINE

01 Foundations of Flow Matching

02 Brownian Motion & Stochastic Differential Equations

03 GRPO & Flow Matching

04 Notebook & References

PART 1

Foundations of Flow Matching

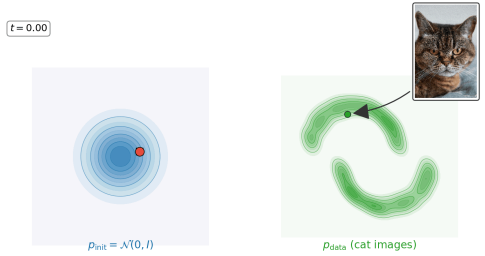
Flow Matching, General Idea

Goal: generate samples from a complex distribution p_{data} (images, videos, molecules, …) that we only access through examples

Idea: learn to *continuously transport* a simple distribution (e.g. Gaussian noise) into the data distribution over time $t \in [0, 1]$

$$\underbrace{p_0 = \mathcal{N}(0, I)}_{\text{easy to sample}} \xrightarrow{\text{learned transport}} \underbrace{p_1 = p_{\text{data}}}_{\text{what we want}}$$

Why learn a velocity instead of the map itself? We could try to learn the noise-to-data map directly, but it's globally complex. One function would have to encode the entire transformation. Learning a vector field instead means the network answers only a *local* question: where to move from *this* point at *this* time. The global transformation then emerges by integrating many easy local decisions.



To make this precise, we need to talk about how points *move* in space over time. The next few slides define **ODEs, vector fields, flows**, then we return to flow matching itself.

Ordinary Differential Equations

DEFINITION

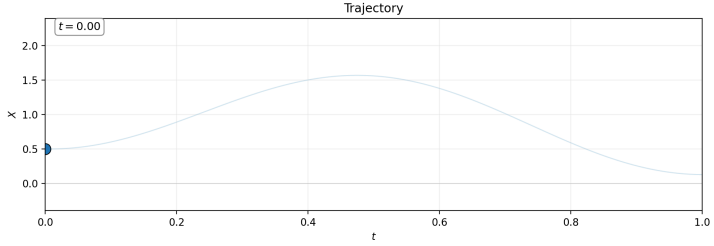
An **ODE** is an equation connecting an unknown function $X : I \to \mathbb{R}^d$ to its derivatives, where $I \subseteq \mathbb{R}$ is an interval. A **first-order ODE** has the form

$$\frac{dX_t}{dt} = f(X_t, t), \qquad f : \mathbb{R}^d \times I \to \mathbb{R}^d.$$

DEFINITION

A **solution**, called a **trajectory**, is a differentiable function $X : I \to \mathbb{R}^d,\ t \mapsto X_t$ that satisfies the ODE pointwise.

Why first-order? In flow matching we specify velocities directly; that is what the neural network outputs, so first-order is the right level of description. We take $I = [0, 1]$ and interpret t as time



EXAMPLE

A water particle flowing downstream: at each time t, X_t gives its position in the river.

Vector Field

DEFINITION

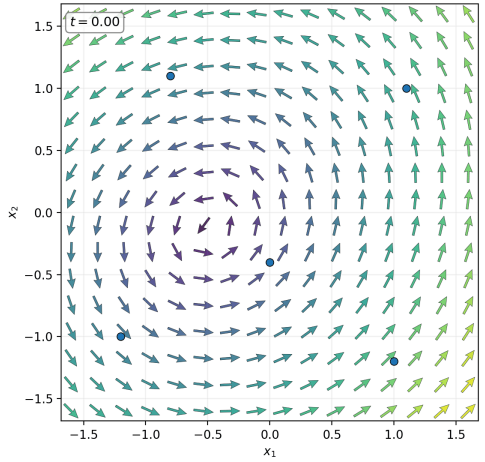
A **vector field** is a function that, for every time t and location x, returns a vector $u_t(x) \in \mathbb{R}^d$ specifying a velocity in space:

$$u : \mathbb{R}^d \times I \to \mathbb{R}^d, \qquad (x, t) \mapsto u_t(x).$$

Every (sufficiently regular) vector field defines an **ODE**

Paired with an initial condition, it constitutes an **initial value problem (IVP)**:

- $\dot{X}_t = u_t(X_t)$: the trajectory follows the field
- $X_0 = x_0$: the trajectory starts at an initial point



THEOREM

Picard–Lindelöf. If u_t is Lipschitz in x and continuous in t, the IVP has a unique solution for every x_0. Neural networks satisfy this condition in practice, so we'll assume it without further comment

Flow

DEFINITION

A flow is a one-parameter family of functions $\{\Psi_t\}_{t\in[0,1]}$, where each $\Psi_t : \mathbb{R}^d \to \mathbb{R}^d$ maps a starting point to its position at time t

Equivalently, the family folds into a single combined map:

$$\Psi : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d, \qquad (x_0, t) \to \Psi_t(x_0)$$

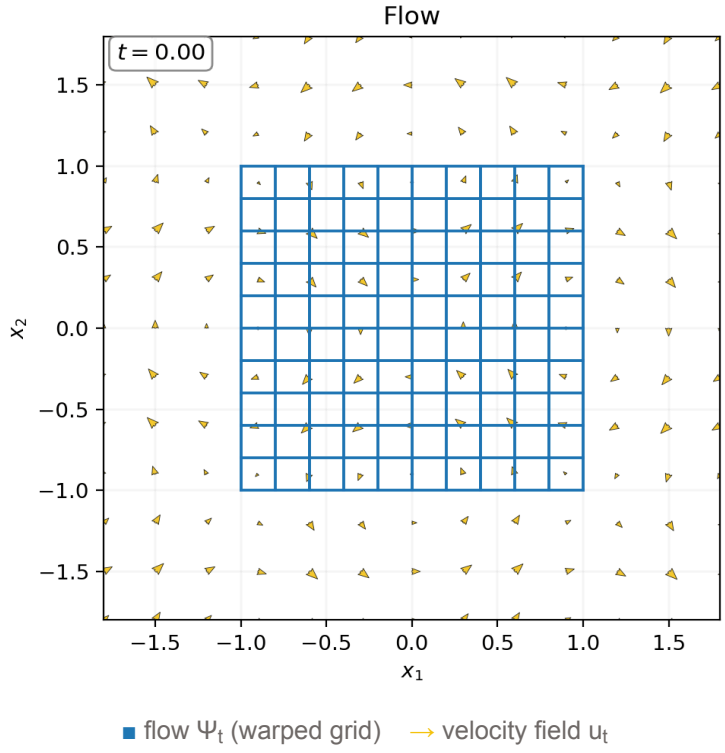
The flow is defined by an ODE: for every point x_0:

$$\frac{d}{dt}\Psi_t(x_0) = u_t(\Psi_t(x_0)), \qquad \Psi_0(x_0) = x_0$$

So the flow is not a single solution but all of them at once, one trajectory per each starting point. Fixing x_0 recovers an initial trajectory, $X_t = \Psi_t(x_0)$ with $X_0 = x_0$

The flow also transports distributions, not just points:

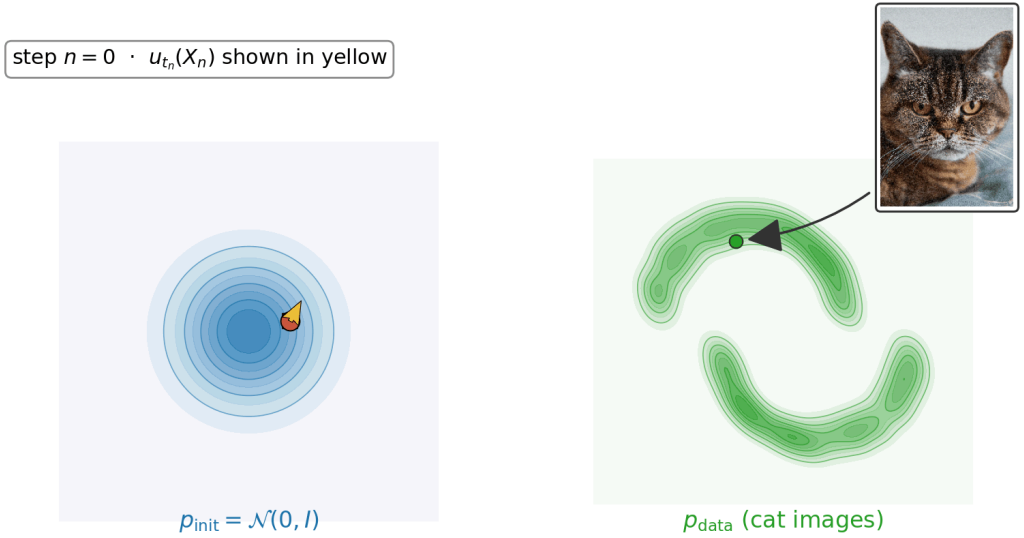
If $X_0 \sim p_0$ then $X_t = \Psi_t(X_0)$ has distribution p_t. So, a vector field u_t defines an entire path of distributions $\{p_t\}_{t\in[0,1]}$ interpolating between p_0 and p_1



From Flows to Flow Matching

In **flow matching** we re-use the machinery above to transport noise into data on $I = [0, 1]$:

- A **trajectory** X_t is the path of a single noisy sample as it is transformed into a data sample
- The **vector field** u_t is what we learn: a neural network trained to point toward data
- The **flow** Ψ_t carries *every* noise point to a data point at once; it is the generative map



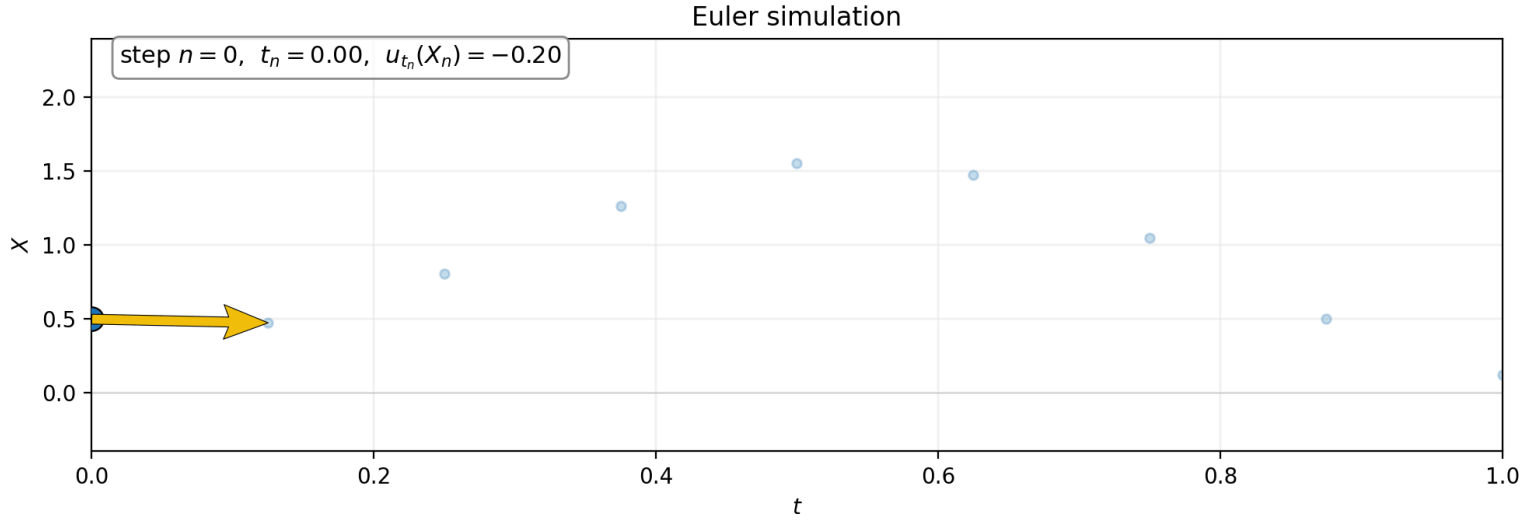
Sampling: Simulating an ODE

In general it is intractable to compute the flow explicitly (unless u_t is very simple, e.g. constant)

To simulate the ODE we use numerical methods. The simplest and one of the most effective is the **Euler method**:

$$X_{t+h} = X_t + h \cdot u_t(X_t), \qquad t \in \{0, h, 2h, \dots, (N-1)h\}$$

where $N \in \mathbb{N}$ is the number of simulation steps and $h = \frac{1}{N}$ is the step size.



Key property for RL: randomness comes only from the initial noise $x_0 \sim p_0$, every Euler step is deterministic. This will be a problem when we want to *explore*. We'll fix it in the GRPO setup by injecting stochasticity as each step

Interpolation Schedules & Training

We need paths connecting noise $x_0 \sim p_0$ to data $x_1 \sim p_{\text{data}}$. A general construction writes $x_t = \alpha_t\, x_0 + \beta_t\, x_1$, with the schedule (α_t, β_t) chosen so that x_0 is pure noise and x_1 is pure data. Two common choices:

- **Linear**: $\alpha_t = 1 - t,\ \beta_t = t$ (straight line from x_0 to x_1).
- **Variance-preserving**: $\alpha_t = \cos(\frac{\pi t}{2}),\ \beta_t = \sin(\frac{\pi t}{2})$.

Each point x_t has a well-defined velocity $u_t(x_t \mid x_0, x_1) = \dot{\alpha}_t\, x_0 + \dot{\beta}_t\, x_1$. For the linear schedule, $\dot{\alpha}_t = -1$ and $\dot{\beta}_t = 1$, so this simplifies to $x_1 - x_0$.

For the rest of this lecture we use linear interpolation

To train, sample $x_0 \sim p_0,\ x_1 \sim p_{\text{data}},\ t \sim \mathcal{U}[0,1]$, form x_t, and regress the network's predicted velocity onto the path velocity:

$$\mathcal{L}(\theta) = \mathbb{E}\big[\, \|u_t^\theta(x_t) - (x_1 - x_0)\|^2 \,\big]$$

Intuition: Each example sees only one noise–data pair, but in expectation the network learns the *marginal* velocity at x_t — the average direction transporting noise into data. This gives a good generative model, but the target $x_1 - x_0$ knows nothing about reward or preference

TERMINOLOGY

In the diffusion literature this is usually called a **noise schedule**, parameterized as $x_t = \alpha_t\, x_1 + \sigma_t\, \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$.

PART 2

Brownian motion & Stochastic Differential Equations (SDE)

From ODE to SDE in Practice

We discretise the SDE with the **Euler–Maruyama** scheme. Comparing one step of each:

$$\underbrace{X_{t+h} = X_t + h\, u_t^\theta(X_t)}_{\text{ODE (deterministic)}} \qquad \longrightarrow \qquad \underbrace{X_{t+h} = X_t + h\, b_t^\theta(X_t) + \sigma\sqrt{h}\,\varepsilon_t}_{\text{SDE (stochastic)}}$$

with $\varepsilon_t \sim \mathcal{N}(0, I)$ drawn **independently at every step**.

- The noise scale σ controls exploration: $\sigma \to 0$ recovers the ODE; larger σ gives noisier, more diverse rollouts.
- The drift b_t^θ is the **learned velocity field corrected by the score** so that the marginals p_t are preserved:

$$b_t^\theta(x) \;=\; u_t^\theta(x) \;+\; \tfrac{\sigma^2}{2}\,\nabla_x \log p_t(x)$$

- For **flow** with linear interpolant $X_t = (1-t)X_0 + tX_1$ ($X_0 \sim \mathcal{N}(0, I)$ noise, $X_1 \sim p_{\text{data}}$), the score can be re-expressed purely in terms of u_t^θ, giving the closed-form drift (see Appendix A):

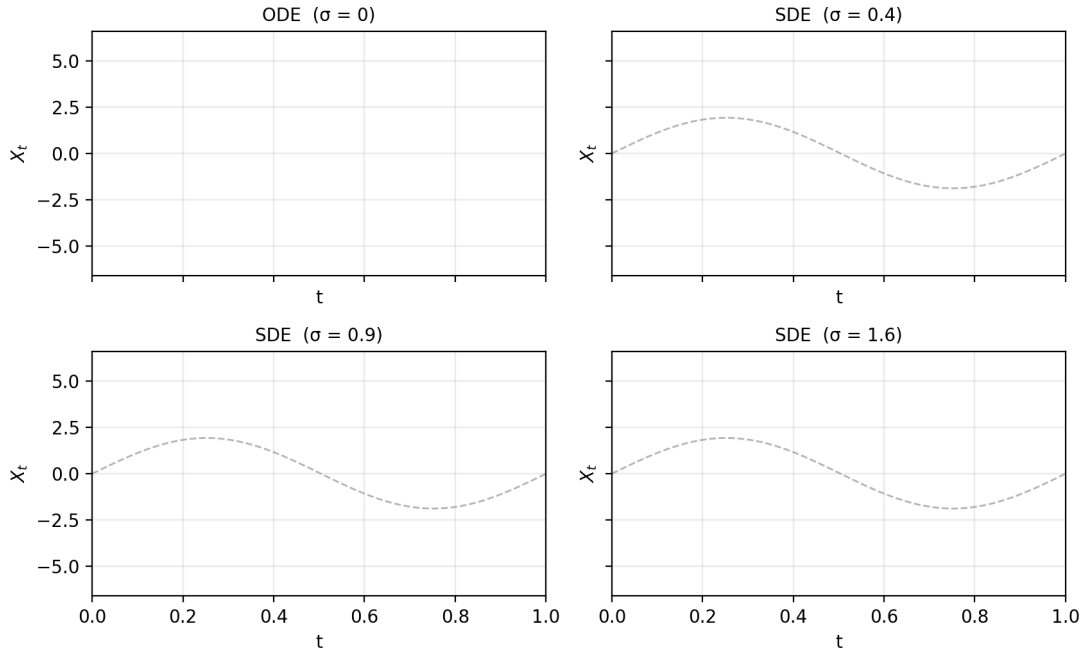
$$b_t^\theta(x) \;=\; u_t^\theta(x) \;+\; \frac{\sigma_t^2}{2(1-t)}\,\big(t\,u_t^\theta(x) - x\big)$$

- Each Euler step is therefore a **Gaussian transition**:

$$\pi_\theta(X_{t+h} \mid X_t) = \mathcal{N}\big(X_t + h\, b_t^\theta(X_t),\; \sigma^2 h\, I\big)$$

This is exactly the policy we will optimise with GRPO: a sequence of N Gaussian actions, one per integration step.

ODE vs SDE: where exploration comes from



PART 3

GRPO & Flow Matching

GRPO refresher

A **critic-free PPO variant**: the baseline comes from a *group* of rollouts that share the same prompt, not from a learned value function.

Group sampling. For prompt s, draw G completions from the *ref* policy and score each with a verifier:

$$a_i \sim \pi_{\theta_{\text{ref}}}(\cdot \mid s), \qquad r_i = r_{\text{ver}}(s, a_i), \qquad i = 1, \dots, G.$$

The **group-relative advantage** $\hat{A}_i$ rescales each reward by the group mean and std (formula on the next-to-last slide).

Token ratio. For each token $a_{i,t}$ of completion a_i,

$$\rho_{i,t}(\theta) \;:=\; \frac{\pi_\theta(a_{i,t} \mid s,\, a_{i,<t})}{\pi_{\theta_{\text{ref}}}(a_{i,t} \mid s,\, a_{i,<t})}.$$

Full GRPO objective — PPO-clipped surrogate plus a KL anchor to a frozen reference π_{ref}:

$$\max_\theta \;\; \mathbb{E}_{s,\, a_{1:G} \sim \pi_{\theta_{\text{ref}}}} \left[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{T_i} \sum_{t=1}^{T_i} \min\!\Big(\rho_{i,t}(\theta) \hat{A}_i,\; \text{clip}\big(\rho_{i,t}(\theta),\, 1-\epsilon_{\text{clip}},\, 1+\epsilon_{\text{clip}}\big) \hat{A}_i \Big) \right] \;-\; \beta\, \hat{D}_{\text{KL}}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big)$$

- Above-baseline siblings get $\hat{A}_i > 0$ and are pushed up; below-baseline ones get $\hat{A}_i < 0$ and are suppressed.
- Clipping at radius ϵ_{clip} stops the per-token ratio from drifting too far in one update — the standard PPO safeguard.
- The KL penalty anchors to π_{ref} (a fixed snapshot), **not** to $\pi_{\theta_{\text{ref}}}$ — the same pattern as RLHF.

GRPO with Gaussian Policies $\Rightarrow$ Closed-Form KL

Policy at each step is Gaussian with covariance $\sigma_t^2 h\, I$ **that does not depend on** θ. Only the *mean* moves with θ

With $\mu_t^\theta = X_t + h\, b_t^\theta(X_t)$ and $\mu_t^{\text{ref}} = X_t + h\, b_t^{\text{ref}}(X_t)$

The per-step log-ratio reduces to a difference of squared distances:

$$\log \frac{\pi_\theta(X_{t+h} \mid X_t)}{\pi_{\text{ref}}(X_{t+h} \mid X_t)} \;=\; \frac{\|X_{t+h} - \mu_{\text{ref}}\|^2 \;-\; \|X_{t+h} - \mu_\theta\|^2}{2\sigma_t^2 h}$$

Taking the expectation $\mathrm{KL} = \mathbb{E}_{X\sim\pi^\theta}[\log r_t(X)]$ results in the **closed form** (see Appendix B):

$$\mathrm{KL}\big(\pi^\theta(\cdot \mid X_t) \,\|\, \pi^{\text{ref}}(\cdot \mid X_t)\big) \;=\; \frac{\|\mu_t^\theta - \mu_t^{\text{ref}}\|^2}{2\sigma_t^2 h}.$$

Substituting $\mu_t = X_t + h\, b_t$ and the flow drift $b_t = u_t + \frac{\sigma_t^2}{2(1-t)}(t\, u_t - x)$, the $-x$ score-correction term **cancels in the difference**:

$$\mu_t^\theta - \mu_t^{\text{ref}} \;=\; h\,\big(b_t^\theta(X_t) - b_t^{\text{ref}}(X_t)\big) \;=\; h\, C_t\,\big(u_t^\theta(X_t) - u_t^{\text{ref}}(X_t)\big), \qquad C_t \;\triangleq\; 1 + \frac{t\,\sigma_t^2}{2(1-t)}.$$

So the per-step KL reduces to a **time-dependent** weight times the squared residual of the learned velocity:

$$\mathrm{KL}\big(\pi^\theta(\cdot \mid X_t) \,\|\, \pi^{\text{ref}}(\cdot \mid X_t)\big) \;=\; \frac{h\, C_t^2}{2\sigma_t^2}\,\big\|u_t^\theta(X_t) - u_t^{\text{ref}}(X_t)\big\|^2.$$

GRPO with Gaussian Policies $\Rightarrow$ Closed-Form KL (cont.)

Group-relative advantage (recall, G rollouts scored by the verifier):

$$\hat{A}_i \;=\; \frac{r_i - \bar{r}_s}{\hat{\sigma}_s + \epsilon_{\text{std}}}, \qquad \bar{r}_s = \tfrac{1}{G}\sum_{i=1}^{G} r_i, \qquad \hat{\sigma}_s^2 = \tfrac{1}{G}\sum_{i=1}^{G}(r_i - \bar{r}_s)^2.$$

Time-dependent KL weight (recall):

$$C_t \;\triangleq\; 1 + \frac{t\,\sigma_t^2}{2(1-t)}.$$

Aggregating the per-step terms across all steps of the sampled trajectory, the objective becomes:

$$\mathcal{L}_{\text{GRPO}} = \mathbb{E}\Big[\sum_k \min\big(r_{t_k}\hat{A},\; \text{clip}(r_{t_k}, 1{-}\epsilon, 1{+}\epsilon)\hat{A}\big)\Big] \;-\; \beta \sum_k \frac{h\,C_{t_k}^2}{2\sigma_{t_k}^2}\big\|u_{t_k}^{\theta}(X_{t_k}) - u_{t_k}^{\text{ref}}(X_{t_k})\big\|^2$$

Notebook to Practice

1 **Build the pipeline, end to end**

Train flow matching on a 2D checkerboard distribution, swap the ODE for an SDE, and let GRPO steer it with a predefined reward

2 **Design your custom reward**

Smooth, sparse, multi-modal, repulsive. See how reward shapes the policy and find out what the policy can (and can't) learn

3 **Sweep across KL coefficients**

Watch the trade-off between maximizing the reward and staying close to the prior

References

- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., Guo, D. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv:2402.03300. arxiv.org/abs/2402.03300
- Liu, J., Liu, G., Liang, J., Li, Y., Liu, J., Wang, X., Wan, P., Zhang, D., Ouyang, W. (2025). *Flow-GRPO: Training Flow Matching Models via Online RL*. arXiv:2505.05470. arxiv.org/abs/2505.05470
- Holderrieth, P., Erives, E. (2026). *Introduction to Flow Matching and Diffusion Models*. arXiv:2506.02070. diffusion.csail.mit.edu

PART A

Appendix

Appendix A: The Flow Drift b_t^θ

Recall. $b_t^\theta(x) = u_t^\theta(x) + \frac{\sigma_t^2}{2}\nabla_x \log p_t(x)$.

Setup. Gaussian mixture path $X_t = \alpha_t X_0 + \beta_t X_1$ with $X_0 \sim \mathcal{N}(0, I)$, $X_1 \sim p_{\text{data}}$, and boundary $(\alpha_0, \beta_0) = (1, 0)$, $(\alpha_1, \beta_1) = (0, 1)$. Conditioning on the data endpoint gives a tractable Gaussian:

$$p_t(x \mid x_1) \;=\; \mathcal{N}\big(\beta_t\, x_1,\; \alpha_t^2 I\big).$$

Step 1. Differentiating the interpolant and conditioning on $X_t = x$:

$$u_t(x) \;=\; \dot\alpha_t\, \mathbb{E}[X_0 \mid X_t = x] \;+\; \dot\beta_t\, \mathbb{E}[X_1 \mid X_t = x].$$

Using the consistency constraint $\alpha_t\mathbb{E}[X_0|X_t] + \beta_t\mathbb{E}[X_1|X_t] = x$ to eliminate $\mathbb{E}[X_0|X_t]$:

$$u_t(x) \;=\; \tfrac{\dot\alpha_t}{\alpha_t}\, x \;+\; \tfrac{\alpha_t\dot\beta_t - \beta_t\dot\alpha_t}{\alpha_t}\, \mathbb{E}[X_1 \mid X_t = x]. \qquad (\star)$$

Step 2. Differentiate the conditional Gaussian: $\nabla_x \log p_t(x|x_1) = -(x - \beta_t x_1)/\alpha_t^2$. Marginalising via $\nabla \log p_t(x) = \mathbb{E}_{X_1|X_t=x}[\nabla \log p_t(x|X_1)]$ (differentiation under the integral + Bayes):

$$\nabla_x \log p_t(x) \;=\; \frac{\beta_t\, \mathbb{E}[X_1 \mid X_t = x] \;-\; x}{\alpha_t^2}. \qquad (\star\star)$$

Steps 1 and 2 give us u_t and $\nabla_x \log p_t$ each as an affine function of the *same* unknown $\mathbb{E}[X_1 \mid X_t = x]$. Continued on the next slide.

Appendix A (cont.): The Flow Drift $b_t^\theta$$

Step 3. Solving $(\star\star)$ for $\mathbb{E}[X_1|X_t]$ and substituting into $(\star)$ gives a relation for *any* Gaussian mixture path:

$$\boxed{\nabla_x \log p_t(x) \;=\; \frac{\beta_t\, u_t(x) \;-\; \dot\beta_t\, x}{\alpha_t\,(\alpha_t\dot\beta_t - \beta_t\dot\alpha_t)}}.$$

This relationship between score function and vector field is true for any choice of the schedulers α_t and β_t.

Step 4. Plug in $\alpha_t = 1-t,\ \beta_t = t$ (so $\dot\alpha_t = -1,\ \dot\beta_t = 1,\ \alpha_t\dot\beta_t - \beta_t\dot\alpha_t = 1$):

$$\nabla_x \log p_t(x) \;=\; \frac{t\,u_t^\theta(x) - x}{1-t} \quad\Longrightarrow\quad b_t^\theta(x) \;=\; u_t^\theta(x) \;+\; \frac{\sigma_t^2}{2(1-t)}\big(t\,u_t^\theta(x) - x\big).$$

Appendix B: The closed-form KL

Setup. Two isotropic Gaussian per-step policies with **identical** covariance:

$$\pi_\theta(\cdot \mid X_t) = \mathcal{N}(\mu_t^\theta,\ \Sigma), \qquad \pi_{\text{ref}}(\cdot \mid X_t) = \mathcal{N}(\mu_t^{\text{ref}},\ \Sigma), \qquad \Sigma = \sigma_t^2 h\, I.$$

Step 1. log-ratio. The Gaussian log-density is $\log \mathcal{N}(x;\mu,\Sigma) = -\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu) + \text{const}(\Sigma)$. The Σ-dependent normalizers are *the same* under θ and ref, so they cancel:

$$\log \pi_\theta(X) - \log \pi_{\text{ref}}(X) \ = \ \tfrac{1}{2\sigma_t^2 h}\left[\|X-\mu_t^{\text{ref}}\|^2 - \|X-\mu_t^\theta\|^2\right]$$

Step 2. KL by definition. $\mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}}) = \mathbb{E}_{X\sim\pi_\theta}\left[\log \pi_\theta(X) - \log \pi_{\text{ref}}(X)\right]$. Parametrize π_θ as $X = \mu_t^\theta + \sigma_t\sqrt{h}\,\varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$:

$$X - \mu_t^\theta = \sigma_t\sqrt{h}\,\varepsilon, \qquad X - \mu_t^{\text{ref}} = (\mu_t^\theta - \mu_t^{\text{ref}}) + \sigma_t\sqrt{h}\,\varepsilon$$

Step 3. expand the squared norms. Using $\|a+b\|^2 = \|a\|^2 + 2a^\top b + \|b\|^2$ (the $\sigma_t^2 h\,\|\varepsilon\|^2$ terms cancel):

$$\|X-\mu_t^{\text{ref}}\|^2 - \|X-\mu_t^\theta\|^2 \ = \ \|\mu_t^\theta - \mu_t^{\text{ref}}\|^2 \ + \ 2\sigma_t\sqrt{h}\,(\mu_t^\theta - \mu_t^{\text{ref}})^\top \varepsilon$$

Step 4. take the expectation. $\mathbb{E}[\varepsilon] = 0$, so the cross-term vanishes:

$$\mathbb{E}_{X\sim\pi_\theta}\left[\|X-\mu_t^{\text{ref}}\|^2 - \|X-\mu_t^\theta\|^2\right] \ = \ \|\mu_t^\theta - \mu_t^{\text{ref}}\|^2$$

Dividing by $2\sigma_t^2 h$ recovers the formula on the main slide:

$$\boxed{\mathrm{KL}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big) \ = \ \frac{\|\mu_t^\theta - \mu_t^{\text{ref}}\|^2}{2\sigma_t^2 h}}$$